

Sparse Certified Component Incidence

An output-sensitive, polynomial-time primitive for
compressed unknot-recognition hierarchies

Research contribution prepared for the ProveIt project

8 October 2026

Based on the explicitly pinned repository snapshot below

Abstract

The maintained marked-orbit interface in ProveIt computes a dense array with 2^r entries for r supplied ports, even when only a few incidence signatures occur. We replace Boolean-lattice enumeration with exact recovery from nonnegative subset sums. If s signatures occur, at most rs subset queries, each reducible to one unweighted interval-orbit count, recover all positive entries. The algorithm needs neither a supplied sparsity bound nor an untrusted stopping certificate. A separate verifier checks positive atom masses, dominating zero sets, and conservation of total mass; it does not repeat the recovery algorithm.

For explicit interval-union ports this is more than a sparse-input result. Classical weighted Agol–Hass–Thurston theory implies polynomially many signatures in the binary input size. We give a conservative explicit support bound extracted from its cycle and weight-breakpoint estimates. Consequently, the new unweighted reduction is polynomial for every supplied interval system, with no logarithmic restriction on the number of ports. This is an implementation and certification improvement over the maintained dense interface, not the first polynomial weighted-orbit theorem. We also obtain signature-resolved parity consistency using a double cover and exhibit a four-point obstruction to using incidence alone as a state for future gluing.

The delivery contains source overlays, independent checkers, 36 passing test methods, raw paired benchmarks, and exact dependency snapshots. On three 12-port disjoint workloads the orbit-query count decreases from 4,096 to 79; small full-support controls are slower. These are interval-subsystem results. No whole-knot speedup or general quasi-polynomial unknot-recognition theorem is asserted.

Repository anchor. `VladimirReshetnikov/ProveIt, Topology/UnknotRecognition`.

Pinned commit. `274909dd8724e411ece14e47ac4addea717aeca6`.

Status. Proved local reduction and certificate theorems; implemented research add-on; global hierarchy bounds remain proof obligations.

Contents

1	The improvement and its exact scope	4
1.1	What is being accelerated	4
1.2	Why the result matters without proving the global target	4
1.3	Results and attribution	4
2	Binary interval systems and marked signatures	5
2.1	Parameters and representation costs	5
2.2	The zeta transform	6
3	Computing a zeta value by one unweighted count	6
3.1	The coning construction	6
3.2	Exact reuse and proof binding	7
4	Output-sensitive recovery from nonnegative subset sums	7
4.1	Residual measures and a minimal positive set	7
4.2	Why nonnegativity cannot be omitted	8
4.3	Bookkeeping and bit complexity	8
5	Why interval signatures have polynomial support	9
5.1	The classical weighted-orbit input	9
5.2	A conservative explicit bound	9
5.3	Why arbitrary Boolean measures do not contradict the bound	10
6	A sparse certificate independent of reconstruction	11
6.1	Facts checked for each atom	11
6.2	Discarding proofs of successful intermediate deletions	12
6.3	Binding, independence, and the trust boundary	12
6.4	A worked two-port certificate	12
7	Structured families and exact query counts	13
7.1	Coincident and nested ports	13
7.2	Disjoint ports with singleton signatures	13
7.3	Full-support controls and the absence of a universal speed ratio	14
8	Signature-resolved parity consistency	14
9	Why incidence alone cannot support future gluing	15
10	Implementation, interfaces, and resource semantics	15
10.1	Delivery layout and unchanged dependencies	15

10.2 Examples of use	16
10.3 Limits and interrupted runs	16
10.4 What integration changes	17
11 Validation and paired measurements	17
11.1 Focused tests and their scope	17
11.2 Benchmark protocol	18
11.3 Paired results	18
11.4 Scale, certificates, and representation effects	19
11.5 What the experiments do not establish	19
12 The remaining route to quasi-polynomial recognition	20
12.1 A compositional bound with named obligations	20
12.2 Where the remaining difficulty is located	20
13 Recommended changes to the maintained analysis	21
13.1 Clarify representation rather than retract the dense theorem	21
13.2 Adopt a separate sparse result type	21
13.3 Keep evidence categories separate	21
14 Further research questions and proposed deliverables	21
15 Reproducibility and delivery contents	22
16 Conclusion	23

1 The improvement and its exact scope

1.1 What is being accelerated

The relevant current bottleneck is not an elementary crossing simplification. It is the representation and reconstruction of component information in a binary-encoded interval relation. The inspected synthesis already contains an unweighted Agol–Hass–Thurston orbit kernel, optional local orbit proofs, an independent replay checker, and an exact reduction of marked incidence to coned orbit counts [4, 5, 6]. Its maintained incidence API returns all 2^r subset counts. Caching equal marked unions saves some orbit calls but cannot remove the dense output or Boolean inversion.

This report changes that contract deliberately: the new API returns only positive pairs $(T, h(T))$. Missing masks have count zero. The unmarked signature $T = \emptyset$ is included when its count is positive. No dense array is constructed internally, and the output can be checked without expanding one. This is a companion interface, not a silently incompatible change to the existing dense function.

The mathematical input consists of an explicit list of interval pairings and explicit finite unions of intervals designated as ports. A normal-surface adapter may supply such data, but the add-on does not construct that adapter or establish its connection to a diagram. In particular, the measured workloads below are supplied interval systems, not diagrams on which the entire unknot recognizer was run.

1.2 Why the result matters without proving the global target

A compressed hierarchy must repeatedly answer local questions without expanding the objects represented by large integers. An unnecessary 2^r representation cost is a genuine obstruction when the number of marked pieces grows. Removing it is useful even before the rest of the hierarchy is available. On the other hand, port incidence does not contain attachment maps or even enough information to update arbitrary future identifications. Section 9 makes this limitation a theorem rather than a vague warning.

The external complexity context is also precise. Lackenby’s 2026 preprint provides hierarchy algorithms and a bound on the number of iterations in terms of hierarchy length and pattern complexity. Its Section 9 separates that iteration analysis from the missing cost specifications for some local steps [2]. The present work implements one narrowly specified aggregate primitive. It does not identify that primitive with every missing step in the preprint.

We use “quasi-polynomial” for $\exp((\log n)^{O(1)})$. The specific target $n^{O(\log n)}$ is the smaller form $\exp(O((\log n)^2))$. A polynomial local query does not establish either global bound unless state sizes, iteration counts, other operations, and completeness are also controlled.

1.3 Results and attribution

The main results are as follows, with their dependencies made explicit.

1. A nonnegative Boolean zeta transform with s positive entries is recovered using at most rs oracle calls (Theorem 4.2). The proof uses monotonicity and minimal positive sets.
2. For interval-union ports, those oracle calls are unweighted coned orbit queries (Lemma 3.1). Weighted AHT theory supplies a polynomial bound on s , sharpened here into a conservative explicit expression (Theorem 5.1).
3. A sparse forward certificate establishes every recovered atom and completeness independently of the search decisions (Theorem 6.1). Unneeded search proofs can be discarded.

4. The parity-cover construction resolves consistent and inconsistent components separately for each signature (Theorem 8.1). Incidence-only gluing updates are impossible in general (Proposition 9.1).

The coning identity was already used by the repository. The weighted AHT polynomial theorem is classical [1]. Additive-query reconstruction of weighted hypergraphs is an established subject; for example, Bshouty–Mazzawi study stronger query bounds under constant-rank hypotheses [3]. We make no priority claim for the elementary greedy recovery principle or for polynomial weighted-orbit computation. The contribution is the precise reduction, sparse certificate contract, explicit support accounting, and tested implementation built around the existing unweighted kernel. No formal proof-assistant verification has been performed.

2 Binary interval systems and marked signatures

Definition 2.1. An *interval system* has universe $X_N = \{0, \dots, N-1\}$ and k partial isometries. A pairing is either

$$[a, b] \longrightarrow [c, d], \quad x \longmapsto x + c - a,$$

or

$$[a, b] \longrightarrow [c, d], \quad x \longmapsto a + d - x,$$

where $b - a = d - c$, all endpoints lie in X_N , and intervals are inclusive. The generated equivalence relation is the smallest equivalence relation containing every pair $x \sim g(x)$ on each pairing domain.

Pairings are undirected as generators of equivalence: replacing one by its inverse makes no difference. The supplied list and its order nevertheless matter to the binding of a particular orbit trace. Reflection of interval order is not an orientation character of a surface.

A port A_i is a finite union of half-open intervals $[\ell, u)$ in $[0, N)$. Overlapping and adjacent intervals are normalized into disjoint runs. Ports may be empty, coincide, overlap, or meet the same orbit many times. We write r for their number and M for the total number of supplied interval pieces; normalization cannot increase M . Our mathematical port labels are $[r] = \{0, \dots, r-1\}$, matching the implementation's bit positions.

For an orbit O , define its *signature*

$$\sigma(O) = \{i \in [r] : O \cap A_i \neq \emptyset\}.$$

The desired histogram and its support size are

$$h(T) = \#\{O : \sigma(O) = T\}, \quad s = \#\{T : h(T) > 0\}.$$

If C is the total orbit count, then $\sum_T h(T) = C$. For $N = 0$, all these counts vanish. For $r = 0$ and $N > 0$, the only signature is empty, with mass C .

2.1 Parameters and representation costs

Set

$$B = \max\{1, \lceil \log_2(N+1) \rceil\}, \quad \Lambda = B + r + 1.$$

All component counts fit in B bits, and each signature mask fits in r bits. A coarse explicit input length is polynomially equivalent to $(k + M)B + r$, with delimiters and interval-list indexing included. It is not polynomially equivalent to N when the universe is large.

A dense histogram has 2^r slots, independently of s . A sparse histogram has s positive records, occupying $O(s(B + r))$ bits up to framing overhead. Converting the latter to the former necessarily reintroduces the dense output cost. Consequently, the polynomial claim in this report applies to the sparse API, not to a request for every zero entry explicitly.

2.2 The zeta transform

For $S \subseteq [r]$, let

$$G(S) = \sum_{T \subseteq S} h(T). \quad (1)$$

It counts orbits whose signatures are wholly contained in S . Equivalently, it counts orbits avoiding every port with index outside S . The histogram is a nonnegative measure on the Boolean lattice, and G is its lower zeta transform. A dense inverse transform is one possible reconstruction, but nonnegativity permits a different one.

3 Computing a zeta value by one unweighted count

3.1 The coning construction

Normalize a nonempty marked set A as disjoint half-open intervals $[\ell_j, u_j)$. Choose the left endpoint z of its first interval. For each interval of length at least two, add the translation pairing

$$[\ell_j, u_j - 2] \longrightarrow [\ell_j + 1, u_j - 1].$$

For every other interval, add the singleton pairing $z \sim \ell_j$. These new relations identify all marked points, using fewer than $2m_A$ additional pairings when the normalized union has m_A pieces. They do not enumerate the marked points.

Lemma 3.1 (Coned-count identity). *Let C_A be the orbit count after adjoining the coning pairings for nonempty A , and let $H(A)$ be the number of original orbits meeting A . Then*

$$C_A = C - H(A) + 1.$$

Consequently, with $A = \bigcup_{i \notin S} A_i$,

$$G(S) = \begin{cases} C, & A = \emptyset, \\ C_A - 1, & A \neq \emptyset. \end{cases} \quad (2)$$

Proof. Pass first to the quotient set of original orbits. Each new relation has both endpoints in the marked set, so it can affect only orbits meeting A . The translation pairings connect all points in each marked interval; the singleton star connects the intervals. Therefore the $H(A)$ affected original classes become exactly one class. Every other original class remains unchanged. This proves the first identity. An orbit is counted by $G(S)$ exactly when it avoids A , so $G(S) = C - H(A)$. The empty case requires no coning and must be handled separately. $\square$

The identity is valid for disconnected supports, reflections, static points, and overlapping ports. It does not assume that A is a union of whole original orbits. Coning is an auxiliary query construction, not a claim that the original surface or knot may be modified in this way.

3.2 Exact reuse and proof binding

The implementation keys a query by the actual normalized marked union. Different subset masks producing the same union use one orbit result and, when recording is enabled, one orbit trace. Equality of union counts alone is not used as a reuse criterion. The checker recomputes the union and the coning pairings from caller-supplied ports before validating the trace.

This policy is conservative: distinct singleton unions can produce identical lists of additional pairings. A more general cache keyed by the complete coned input could exploit that fact, but it is not implemented here. In particular, the exact query counts proved in Section 7 describe this union-keyed implementation, not optimal oracle complexity.

Let $P(K, B)$ be a polynomial bit-time bound for the unweighted AHT kernel on K pairings with B -bit endpoints. Each nonempty-union zeta query costs at most $P(k + 2M, B)$ plus normalization. One baseline query obtains C . The existing kernel and its independent local checker are retained unchanged [4, 6].

4 Output-sensitive recovery from nonnegative subset sums

4.1 Residual measures and a minimal positive set

Suppose that exact atoms $(T_1, c_1), \dots, (T_j, c_j)$ have been recovered. Define

$$R_j(S) = G(S) - \sum_{\ell \leq j, T_\ell \subseteq S} c_\ell. \quad (3)$$

If those atoms are correct, R_j is the zeta transform of the remaining nonnegative measure. It is nonnegative and monotone under inclusion.

Lemma 4.1 (Minimal-positive-set lemma). *Let R be the zeta transform of a nonzero nonnegative measure f on $2^{[r]}$. Start with $S = [r]$. Process each bit once, in any fixed order; remove i from the current set whenever $R(S \setminus \{i\}) > 0$. The final set T satisfies*

$$R(T) = f(T) > 0, \quad f(U) = 0 \quad \text{for every } U \subsetneq T.$$

Proof. The current set has positive residual mass initially and after every successful removal. Suppose $U \subsetneq T$ had $f(U) > 0$. Choose $i \in T \setminus U$. That bit was retained when it was tested. At that time, the proposed smaller set contained $T \setminus \{i\}$ and hence U . Its residual mass was therefore at least $f(U) > 0$, so the algorithm would have removed i , a contradiction. No proper subset of T has positive mass. Since $R(T) > 0$, its entire mass is the atom at T . $\square$

Only one pass through the bits is needed. A previously failed deletion need not be retested: later removals only shrink its possible witnesses. This observation is also the basis of the independent zero-set certificate.

Theorem 4.2 (Sparse zeta recovery). *Given the total mass C and an exact oracle for (1), Algorithm 1 returns every positive atom exactly once. If $s = |\text{supp } h|$, it uses at most rs oracle calls, with no prior knowledge of s . The implementation makes exactly r logical queries per recovered atom; implicit values and cached unions may require no new orbit call.*

Proof. Induct on the number of outer iterations. Initially no atoms have been removed, so the residual is G . If the recovered atoms are exact and their mass is less than C , the remaining measure is nonzero.

Algorithm 1: Nonnegative sparse zeta recovery

Input: r , total mass C , exact oracle $G(S)$
Contract: $G(S) = \sum_{T \subseteq S} h(T)$ for $h(T) \in \mathbb{N}$

```

1  $E \leftarrow []$ ;  $m \leftarrow 0$ ;
2 while  $m < C$  do
3    $S \leftarrow [r]$ ;  $v \leftarrow C$ ;
4   for  $i = 0, \dots, r - 1$  do
5      $U \leftarrow S \setminus \{i\}$ ;  $w \leftarrow G(U)$ ;
6      $z \leftarrow w - \sum_{(T,c) \in E, T \subseteq U} c$ ;
7     if  $z > 0$  then
8        $S \leftarrow U$ ;  $v \leftarrow w$ ;
9     else
10      retain  $i$  and optionally record the zero witness  $U$ ;
11    $c \leftarrow v - \sum_{(T,a) \in E, T \subseteq S} a$ ;
12   append  $(S, c)$  to  $E$ ;  $m \leftarrow m + c$ ;
13 return  $E$ 

```

Apply Lemma 4.1 to its zeta transform. The final residual value is exactly the positive mass at the selected set, which has not appeared previously. Subtracting it leaves a nonnegative measure, maintaining the induction hypothesis. Thus exactly one new positive atom is recovered in each iteration. There are s such atoms. After their total mass reaches C , the remaining nonnegative measure has total zero and is identically zero. Each iteration makes at most r calls. $\square$

For $r = 0$, a positive total is recovered as the single empty-set atom with no zeta calls. For $C = 0$, the correct empty output is immediate. Neither case requires a special dense representation.

4.2 Why nonnegativity cannot be omitted

The stopping and deletion rules are invalid for a signed measure. For example, with one port, set

$$h(\emptyset) = 1, \quad h(\{0\}) = -1.$$

The total mass is zero although the measure is not zero. Cancellation can also make a proper-set query vanish despite containing a nonzero atom. Our interval histogram is genuinely nonnegative, since its entries count equivalence classes. The generic recovery helper documents this as an oracle contract; it is not a verifier for an arbitrary signed oracle.

4.3 Bookkeeping and bit complexity

Let q be the number of distinct nonempty coned unions actually queried. Then $q \leq rs$, and the total number of orbit calls is at most $1 + rs$. A straightforward residual evaluation scans the previously recovered list. Across all logical queries this takes $O(rs^2\Lambda)$ bit work. Selecting and sorting a marked union has a coarse bound $O((r + M \log(M + 1))\Lambda)$, accounting also for large bit masks.

To avoid hiding a probabilistic dictionary assumption, even the crude cost of comparing each queried

union against every previous key gives the deterministic polynomial bound

$$\begin{aligned} T \leq & (q+1)P(k+2M, B) \\ & + O((rs+1)(r+M\log(M+1)+s)\Lambda + (rs+1)q(M+1)B). \end{aligned} \quad (4)$$

The supplied Python code uses dictionaries and is usually substantially faster than this comparison bound. A sorted dictionary can be substituted without changing the polynomial conclusion. The expression is intentionally conservative; it is not an optimized exponent for integer arithmetic.

The output takes $O(s(B+r))$ bits. Cached unions and counts require $O(q(M+1)B)$ additional bits. Proof recording adds the native orbit traces and witness references. During production, all queried traces can be present before unused traces are pruned, so peak recording storage is not bounded by the size of the final pruned certificate alone.

5 Why interval signatures have polynomial support

5.1 The classical weighted-orbit input

For each $x \in X_N$, attach the nonnegative vector

$$w(x) = (\mathbf{1}_{A_0}(x), \dots, \mathbf{1}_{A_{r-1}}(x)).$$

There is a partition into at most $m = 2M + 1$ constant-vector intervals. For $r = 0$, one may append a dummy zero coordinate, or handle the one-signature case directly. The weight sum on an orbit has positive coordinates exactly at its signature. Classical weighted AHT computes orbit-weight information as a compressed list of constant-weight runs with multiplicities, rather than enumerating all represented orbits [1, Section 6]. Taking positive-coordinate supports of those records and coalescing equal supports therefore gives a polynomial-size incidence list. This already yields a polynomial algorithm for the problem; the purpose here is to realize the same conclusion through the existing unweighted interface and its proof checker.

5.2 A conservative explicit bound

For clarity, we extract a numerical bound rather than simply referring to an unspecified polynomial. We use the following two estimates from the classical AHT analysis: the unweighted schedule has at most $5k^2(2 + \log_2 N)$ cycles for $k \geq 1, N \geq 1$, and one weighted transfer increases the number of constant-weight intervals by at most four [1, proof of Theorem 12 and Lemma 15]. Static contractions output constant-weight pieces; the weighted procedure uses the same schedule.

Theorem 5.1 (Quantitative support bound). *Let $N \geq 1$ and put*

$$Q = 1 + 5k^2(B + 2).$$

For an interval system with M port pieces and r ports,

$$s \leq \min\{N, 2^r, Q(2M + 4Q + 2k + 2)\}. \quad (5)$$

For $k = 0$ the sharper elementary estimate $s \leq 2M + 1$ holds. These are support bounds, not bounds on the number of component orbits C .

Proof. The first two bounds are immediate. For the third, use a classical weighted AHT execution on the indicator-vector weights above. The padded value Q bounds the number of cycles, including the harmless zero-pairing terminal case. Initially there are at most $2M + 1$ constant-vector intervals. Weighted transfers occur at most once per cycle, so the active weight partition has at most

$$m_* = 2M + 1 + 4Q$$

intervals throughout. Static contractions do not increase the number of active constant-weight runs: they delete pieces and concatenate the surviving coordinate intervals. The other non-transfer operations do not alter the weights.

At a static-contraction stage, the complement of all pairing domains and ranges has at most $2k + 1$ intervals, since the list still contains at most k pairings. Intersecting those gaps with a partition into at most m_* constant-weight intervals produces at most $m_* + 2k + 1$ nonempty pieces. This deliberately loose bound follows by sweeping the two lists of endpoints; a partition boundary or gap boundary can start at most one new intersection piece. Each piece contributes at most one constant-weight output record, labelled with its multiplicity. There are at most Q stages, so the number of output records is at most

$$Q(m_* + 2k + 1) = Q(2M + 4Q + 2k + 2).$$

Each final orbit belongs to one of these records and has that record's weight vector. Taking the set of positive coordinates cannot increase the number of distinct labels. The number s of signatures is therefore at most the record count.

When $k = 0$, every point is an orbit and signatures change only at the $2M$ possible port endpoints. At most $2M + 1$ signatures occur directly. $\square$

The bound is independent of the number of empty or duplicate port labels except through the length needed to write each mask. It is not expected to be sharp: the proof multiplies a worst-case active partition size by a worst-case number of contraction stages. It counts output records before coalescing repeated weights, and then counts signatures before coalescing weights with the same support.

Corollary 5.2 (Polynomial marked incidence via unweighted AHT). *Exact sparse component incidence for explicit interval-union ports is computable in deterministic time polynomial in k, M, r, B , using only an unweighted AHT orbit counter. No condition of the form $r = O(\log N)$ or $r = O((\log n)^2)$ is required.*

Proof. Substitute (5) and $q \leq rs$ into (4). Every factor is polynomial in the stated parameters. The case $N = 0$ is immediate. $\square$

Remark 5.3 (What has and has not been strengthened). The repository's dense complexity estimate remains correct for its dense output. The stronger conclusion concerns a different, sparse interface. It follows partly from classical weighted theory that the old synthesis already identifies as an alternative direction. Thus the change removes an implementation bottleneck, not a previously unknown lower bound on the mathematical marked-orbit problem.

5.3 Why arbitrary Boolean measures do not contradict the bound

An arbitrary nonnegative measure can have 2^r positive atoms. The sparse oracle theorem alone does not promise polynomial time in r for that class. Realizing all such signatures as static interval points requires many changes of the indicator vectors. For example, label 2^r consecutive points by their binary expansions and let port i contain the points with bit i set. Every signature occurs, but the explicit port-interval input already grows exponentially. Theorem 5.1 concerns the full interval input, not just the number of port names.

6 A sparse certificate independent of reconstruction

6.1 Facts checked for each atom

A count obtained from a search routine is not yet an independently checked answer. We therefore distinguish the recovery producer from a forward checker. Its input is the original interval system and ports, a baseline orbit proof for C , a list of coned orbit proofs, and an ordered list of claimed positive atoms.

For the j th claimed atom (T_j, c_j) , let

$$K_j(S) = \sum_{\ell < j, T_\ell \subseteq S} c_\ell.$$

The checker requires the following facts.

1. The mask T_j has not appeared earlier, $c_j > 0$, and a certified zeta value satisfies $G(T_j) = K_j(T_j) + c_j$.
2. For each $i \in T_j$, there is a certified set $U_{j,i}$ such that

$$i \notin U_{j,i}, \quad T_j \setminus \{i\} \subseteq U_{j,i}, \quad G(U_{j,i}) = K_j(U_{j,i}). \quad (6)$$

3. After all atoms, $\sum_j c_j = C$, and the declared sparse histogram is exactly the sorted list of those atoms.

A zeta value whose complementary marked union is empty is implicit: it is C and needs no extra proof. Otherwise, the checker derives it as $C_A - 1$ from the local orbit trace on the explicitly reconstructed coned system.

Theorem 6.1 (Soundness and completeness of sparse replay). *If the baseline and coned orbit proofs are valid for the caller's source system, and the atom facts above hold, then the declared sparse histogram is exactly h . The greedy recovery algorithm supplies such a certificate for every complete run. Verification does not require a supplied upper bound on s , a dense zeta transform, or replay of the deletion decisions.*

Proof. Assume inductively that all atoms before j are exact. Subtracting those atoms from h leaves a nonnegative residual measure f_j . Equation (6) says its total mass on every subset of $U_{j,i}$ is zero. Because the summands are nonnegative, each of those atoms has mass zero individually.

Take any proper subset $V \subsetneq T_j$. There is an $i \in T_j \setminus V$, so $V \subseteq T_j \setminus \{i\} \subseteq U_{j,i}$. Thus $f_j(V) = 0$. The certified value at T_j now gives

$$f_j(T_j) = G(T_j) - K_j(T_j) = c_j.$$

This establishes the induction step. At the end, the residual total mass is $C - \sum_j c_j = 0$; its nonnegativity forces every omitted atom to vanish.

For a complete greedy run, take the zeta value at its final set as the support-value witness. For every retained bit, use the set tested at its failed deletion. Later removals guarantee the containment in (6), and the failed test establishes the zero equality. The producer's final mass is C by Theorem 4.2. $\square$

The checker allows any atom ordering and zero sets satisfying these conditions. It does not insist on the producer's preferred bit order or particular greedy path. This separates mathematical validity from search strategy and makes later producer optimizations easier to audit.

6.2 Discarding proofs of successful intermediate deletions

A successful intermediate deletion witnesses positive residual mass in a set that may later shrink. The final forward proof does not need that intermediate fact. It needs only the final support value and the failed removal sets of retained bits. After production, the implementation removes all coned traces not used in those roles and remaps references.

Every retained support-value query was already made during the search, unless it is the implicit value C . Every retained zero-set query was also made during the search. Hence pruning adds no orbit calls, and the resulting certificate contains at most rs distinct coned traces plus its baseline trace. The atom/zero metadata occupies at most $O(sr(r + \log(q + 2)) + sB)$ bits. Trace storage remains polynomial because the AHT schedule produces polynomially many locally checkable operations.

This pruning is more than a size optimization: it removes dependence on facts that are irrelevant to the mathematical conclusion. It does not make an invalid trace valid, and it does not allow incomplete production to publish a partial certificate as a completed histogram.

6.3 Binding, independence, and the trust boundary

The certificate format is named `sparse-incidence-v1`. Each stored union trace is accompanied by the normalized union to which it belongs. Every reference is checked against the union recomputed from its mask and the caller's ports. A trace cannot be rebound to a different union under this format. Duplicate assignments to the same union and unused proof records are rejected. The baseline and each coned trace bind the ordered, canonical pairing list through the existing independent orbit verifier.

The new forward checker imports neither the sparse producer nor its zeta recovery helper. It also does not import the orbit producer. It shares integer decoding, port normalization, and the explicit coning construction; these shared geometric routines are covered by comparisons with literal finite graphs. Its independence claim is therefore structural, not a claim that every line of validation code was written twice.

The native orbit checker validates local reductions, not the scheduling policy used to find them. Accordingly, a complete valid alternative orbit trace can be accepted. An unfinished trace, a changed source binding, or a wrong claimed count is rejected. Large integers can be transported as explicit hexadecimal strings; Boolean values are not accepted as integer fields. The source-to-geometry relationship remains outside the certificate: correct incidence for arbitrary input intervals is not a certificate that those intervals describe a particular knot exterior.

6.4 A worked two-port certificate

Let $N = 5$, with no original pairings, $A_0 = \{0\}$, and $A_1 = \{1\}$. Then

$$G(\emptyset) = 3, \quad G(\{0\}) = G(\{1\}) = 4, \quad G(\{0, 1\}) = 5.$$

The producer first reaches the empty set and outputs mass 3. With that atom removed, it next outputs $\{1\}$ with mass 1, retaining bit 1 because the residual value at the empty set is zero. Finally, it outputs $\{0\}$ with mass 1. For this last atom, the failed deletion of bit 0 was tested at $U = \{1\}$, whose value 4 equals the already recovered mass $3 + 1$. That larger zero set dominates $T \setminus \{0\} = \emptyset$ and is sufficient; there is no need to rerun the earlier search. The total $3 + 1 + 1 = 5$ certifies that no signature is missing.

7 Structured families and exact query counts

7.1 Coincident and nested ports

If all $r \geq 1$ ports equal the same nonempty set A , every orbit has either empty signature or full signature. There is one distinct nonempty marked union, so the implementation uses exactly two orbit calls: the baseline and one coned count. The old cached dense interface can also use two orbit calls, but still allocates 2^r output entries and performs its Boolean array operations. The sparse output has at most two records.

If the ports form a nonempty nested chain

$$A_0 \subseteq A_1 \subseteq \cdots \subseteq A_{r-1},$$

every signature is a final segment of the port order, or empty. Thus $s \leq r + 1$. Every nonempty union equals one port, giving at most $r + 1$ orbit calls including the baseline. Again, the query count alone does not explain the improvement over cached dense inversion: the avoided representation cost is essential.

7.2 Disjoint ports with singleton signatures

The next family separates query savings from array savings. Assume all ports are nonempty and pairwise disjoint, and assume

$$\text{supp } h = \{\emptyset, \{0\}, \dots, \{r-1\}\}, \quad h(T) > 0 \text{ for all these } T. \quad (7)$$

This is realized by static universes with disjoint marked intervals and some unmarked points. It is also realized by the periodic and reflection families used below.

Proposition 7.1 (Exact union-keyed costs). *Under (7), with the implemented increasing bit order,*

$$\begin{aligned} \text{positive records} &= r + 1, \\ \text{logical zeta queries} &= r(r + 1), \\ \text{orbit calls} &= 1 + \frac{r(r + 1)}{2}, \\ \text{retained coned proofs} &= 2r - 1 \quad (r \geq 1). \end{aligned}$$

The dense union-keyed interface makes 2^r orbit calls, including its baseline.

Proof. The empty signature is recovered first. The remaining singleton atoms are then recovered in order $\{r-1\}, \dots, \{0\}$: the deletion rule can remove a lower bit while a higher remaining singleton supplies positive mass, but cannot remove the last remaining candidate.

The first atom queries complementary port-index prefixes $\{0, \dots, j\}$ for $j = 0, \dots, r-1$. When the target singleton is $\{t\}$, the new complements queried after bit t are

$$\{0, \dots, i\} \setminus \{t\}, \quad t < i \leq r-1.$$

There are r prefixes and $\binom{r}{2}$ sets of the second form. They are all distinct: the largest index determines the right endpoint, and a non-prefix set has a unique missing earlier index. Since the ports are nonempty and disjoint, these are also distinct marked unions. Adding the baseline gives the claimed orbit count. There are $r + 1$ atoms, each with r logical tests.

For a singleton $\{t\}$, the support-value proof uses the complement $[r] \setminus \{t\}$ when it is nonempty. Its unique zero witness uses the prefix $\{0, \dots, t\}$. The empty atom uses the full complement, already one of

those prefixes. For $r \geq 2$, these are r prefixes and r singleton complements, with exactly one overlap. For $r = 1$, the singleton's support value is implicit and only the full marked set has a trace. In both cases there are $2r - 1$ retained coned proofs.

Finally, every subset of disjoint nonempty ports has a distinct union, so the old dense interface queries every nonempty subset in addition to its baseline. $\square$

For $r = 12$, this predicts exactly 79 orbit calls and 23 retained coned proofs, compared with 4,096 dense calls. For $r = 32$, it predicts 529 calls and 63 retained proofs. These exact values are tested and observed; they are not inferred from a fitted timing curve.

7.3 Full-support controls and the absence of a universal speed ratio

Sparse reconstruction can be slower when s is close to 2^r on a small instance. Both methods may ultimately query the same distinct unions, while the sparse method pays for repeated residual scans and dictionary accesses. The full-support benchmark controls realize this situation with $r = 3, 4$. They are important counterexamples to a blanket claim that the sparse routine is always faster. A production dispatcher should distinguish an asymptotic representation guarantee from constant-factor choices on tiny instances.

8 Signature-resolved parity consistency

Attach a label $\epsilon_g \in \{0, 1\}$ to every pairing, separate from its interval-order reversal. A component is *consistent* if there is a binary vertex labelling η satisfying $\eta(gx) = \eta(x) + \epsilon_g$ modulo two on every relation. Equivalently, every closed walk has total parity zero.

Construct a double cover on $X_N \times \{0, 1\}$ with relations

$$(x, e) \sim (gx, e + \epsilon_g).$$

Encode its two sheets as consecutive blocks of length N . Each original pairing gives two interval pairings, with at most one extra endpoint bit. Lift each port to *both* sheets:

$$\tilde{A}_i = (A_i \times \{0\}) \cup (A_i \times \{1\}).$$

Let $b(T)$ be the base incidence histogram and $d(T)$ the lifted histogram.

Theorem 8.1 (Parity-resolved incidence). *For each signature T , the numbers $o(T)$ and $u(T)$ of consistent and inconsistent components are*

$$o(T) = d(T) - b(T), \quad u(T) = 2b(T) - d(T). \quad (8)$$

They are computable and independently certifiable in polynomial bit time from the supplied labelled interval system and explicit ports. The two sparse histograms have the same support.

Proof. On a connected base component, choose a root. If every loop has zero parity, the parity accumulated along a path from the root is independent of the path. It defines a labelling, and the double cover has exactly two components, each projecting bijectively onto the base component. If some loop has odd parity, its lift joins the two lifts of the root. Paths from the root then show that the whole double cover of that component is connected.

Because ports are lifted to both sheets, every lifted component has the same signature as its base component. Thus for each T , $b(T) = o(T) + u(T)$ and $d(T) = 2o(T) + u(T)$, giving (8). The construction

uses $2k$ pairings, at most $2M$ port pieces, and $B + 1$ endpoint bits, so Corollary 5.2 applies twice. The two sparse certificates, plus the direct integer identities, establish the result. $\square$

The signed implementation shares a single cycle allowance and a single orbit-query allowance across the base and cover computations. If either is unfinished, it publishes neither histogram nor certificate. The `max_terms` option limits each of the two sparse histograms separately. Its independent checker reconstructs the cover directly and does not import the producer’s lift routine.

When the parity labels actually encode the orientation character of a surface, consistent and inconsistent mean orientable and nonorientable. For arbitrary labels they mean only parity consistency. In particular, a reflection of an interval with parity zero is not a nonorientability obstruction. Lifting ports to only one sheet would invalidate the signature-wise identities and is not an admissible shortcut.

9 Why incidence alone cannot support future gluing

Proposition 9.1 (A four-point attachment obstruction). *There is no general rule that determines the orbit count after adjoining a specified new interval pairing from only the original port-incidence histogram and that new pairing, even when every old component meets each port in exactly one point.*

Proof. Take $X_4 = \{0, 1, 2, 3\}$ and ports $A_0 = \{0, 1\}$ and $A_1 = \{2, 3\}$. Consider two original partitions:

$$\mathcal{P} = \{\{0, 2\}, \{1, 3\}\}, \quad \mathcal{Q} = \{\{0, 3\}, \{1, 2\}\}.$$

They are generated respectively by the positive and negative pairings $[0, 1] \rightarrow [2, 3]$. Both have precisely two components with signature $\{0, 1\}$ and no other components. Each component meets each port once, so even the corresponding per-port multiplicity vectors agree.

Now adjoin the same positive pairing $[0, 1] \rightarrow [2, 3]$ to both systems. It adds no new identification to $\mathcal{P}$, which still has two orbits. For $\mathcal{Q}$, the old and new pairings connect all four points, giving one orbit. The input histogram and the newly supplied pairing are identical, but the correct answers differ. $\square$

The obstruction is present in a finite combinatorial subproblem; it does not depend on an unproved assertion about 3-manifolds. Consequently, a hierarchy implementation must retain enough information about component identity and attachment maps to distinguish these states. Adding aggregate point multiplicities alone does not resolve the example. Possible richer states include compressed boundary correspondences, marked orbit representatives with certified transport, or suitable simultaneous incidence structures retaining source positions. Their closure and size bounds are separate research questions.

10 Implementation, interfaces, and resource semantics

10.1 Delivery layout and unchanged dependencies

The overlay adds six modules under `fast/fastunknot`:

Module	Role
<code>sparse_zeta.py</code>	Generic nonnegative subset-sum recovery.
<code>sparse_incidence_common.py</code>	Exact integer, interval, and coning schemas.
<code>sparse_incidence.py</code>	Sparse producer with shared query budgets.
<code>sparse_incidence_verify.py</code>	Independent forward certificate checker.
<code>sparse_signed_incidence.py</code>	Base/cover producer and parity arithmetic.
<code>sparse_signed_incidence_verify.py</code>	Independent signed-source and cover replay.

The standalone package includes exact copies of `interval_orbits.py`, `interval_orbit_verify.py`, `interval_incidence.py`, and `integer_codec.py`. Their Git blob identifiers and SHA-256 hashes are checked by the test runner. These copies are reproducibility dependencies, not replacement files in the overlay. The original dense implementation is the benchmark baseline.

The principal unsigned entry point is

```
analyze_sparse_port_incidence(
    size, pairings, ports, *, max_ports=None, max_terms=None,
    max_cycles=None, max_orbit_queries=None,
    record_certificate=False, check=None)
```

Pairings are native objects or explicit inclusive rows `[a,b,c,d,sign]`, with sign `+1` or `-1`. The pairing container is an explicit list or tuple. Ports are lists of half-open interval lists. Successful results contain positive `[mask,count]` rows, the total orbit count, and work statistics. Proof recording adds a certificate.

10.2 Examples of use

```
from fastunknot.sparse_incidence import analyze_sparse_port_incidence
from fastunknot.sparse_incidence_verify import (
    verify_sparse_port_incidence_certificate)

n = 10
pairs = []
ports = [[(0, 3)], [(2, 5)]]
result = analyze_sparse_port_incidence(
    n, pairs, ports, record_certificate=True)
assert result['status'] == 'COMPLETE'
assert result['histogram'] == [[0, 5], [1, 2], [2, 2], [3, 1]]
assert verify_sparse_port_incidence_certificate(
    n, pairs, ports, result['certificate'])
```

For the signed API, a raw pairing has one further entry: its parity. A returned row `[mask, consistent, inconsistent]` records both counts. The independent signed checker receives the original signed list, not a producer-supplied cover that it is asked to trust.

10.3 Limits and interrupted runs

The baseline and all coned calls share one `max_cycles` budget. Unused allowance is passed forward; it is not reset for a new subset. `max_orbit_queries` similarly limits the whole computation, including the baseline. `max_terms` limits the number of recovered atoms. An exhausted work allowance returns `INCONCLUSIVE`, a reason, and work statistics, but no histogram, orbit count, or certificate.

An AHT cycle allowance is not a bound on all bit operations. Union normalization, residual sums, dictionary work, trace serialization, and proof replay have their own costs. The cooperative `check` callback is polled through these combinatorial stages and can enforce a caller’s deadline. Ordinary callback exceptions, including a plain `ValueError`, propagate rather than being treated as a false proof; the implementation’s reserved schema and limit exception types remain internal control signals. No claim of adversarial memory containment is made. Serialized-size limits and process-level resource limits remain external responsibilities.

The default `max_ports=None` removes the old interface’s explicit 12-port allowance from this sparse API. That is not a claim that every huge input is cheap. Runtime still depends polynomially on the explicit input, with potentially large exponents and constants. Likewise, `record_certificate=False` is meaningful: producing and replaying proofs has measured overhead.

10.4 What integration changes

The overlay does not alter ordinary recognition dispatch, the dense API, the Rust implementation, or the old orbit scheduler. Existing callers expecting a dense array should continue using the dense function until they are deliberately changed to sparse semantics. A caller that only needs the positive incidence classes can use the new interface directly. Reproducing the local tests does not establish that the full repository’s maintained suite still passes after unrelated integration changes; that suite must be run in the target checkout.

11 Validation and paired measurements

11.1 Focused tests and their scope

The completed run uses CPython 3.13.5 on Linux x86-64. All 36 focused test methods pass in 1.644 seconds in the recorded run. The suite includes:

- Exhaustive recovery for all 6,654 histograms on $r = 0, 1, 2, 3$ whose coefficients lie in $\{0, 1, 2\}$. It checks the recovered entries, the exact logical-query count, and every forward zero witness.
- One thousand random small interval systems, compared with an independent explicit disjoint-set model. Three hundred are also compared with the unchanged native dense routine. All generated sparse proofs are replayed independently.
- Three hundred random parity-labelled systems, compared with an independent labelled-graph traversal that detects inconsistent cycles without using the double-cover counting formula.
- Exact work-budget boundaries, input normalization, Boolean rejection, changed source bindings, missing or invalid zero witnesses, altered atom masses, duplicate or unused proof records, and truncated orbit traces.
- Large binary instances including $N = 2^{16000}$ with 128 ports, hexadecimal JSON round trips, and replay while producer functions are disabled. Singleton-family query and proof formulas are tested for $r = 1, \dots, 16$.

The large-instance tests do not enumerate N points. Literal graph references are used only for small universes. The random interval tests use seed 261008490, and the parity tests use seed 261008491.

The test count means 36 test methods, not 36 mathematical theorems or a formal proof. Several methods contain hundreds or thousands of cases. The full repository suite reported in the inspected

synthesis was not run here. Regina and SnapPy were not installed in this execution environment, and no new native triangulation or knot-diagram benchmark is asserted. The source hashes, exact test log, and structured summary are included.

11.2 Benchmark protocol

Eight workloads compare four arms: the unchanged dense implementation, the sparse implementation, an identical sparse control, and sparse proof production followed by independent verification. The certified arm includes both production and replay in its timing; JSON serialization is outside that region. The baseline already caches equal unions, so the comparison does not unfairly disable a maintained optimization.

Each paired workload has one warmup round and seven measured rounds. Arm order is shuffled using seed 261008492. Tiny cases use batches of identical calls, with per-call times reported. Three additional scale workloads compare only sparse and certified arms, with one warmup and three measured rounds. There are 242 measured arm-round records and 38 warmup records, representing 4,354 measured calls and 630 warmup calls after batching. No timed call in the final benchmark is incomplete.

Correctness comparisons take place outside the timed region. Static cases are checked using an independent endpoint sweep. The period-17 and reflection cases are checked against their explicit residue-class and reflection-orbit formulas. The benchmark preserves every measured sample, its order, repetition count, orbit calls, and orbit cycles. The output is not a best-of-run selection.

An earlier benchmark attempt used a larger 64-port disjoint scale case and was stopped by the execution harness after its 200-second allowance. Only a partial textual log was available; no complete sample set for that scale case was produced. That log is retained separately and is not mixed into the final timing tables. It establishes no timing ratio for the unfinished case. The final disjoint scale workload has 32 ports.

11.3 Paired results

Table 1 reports medians in milliseconds. The ratio column is the median, across matched rounds, of dense time divided by sparse time; it is not necessarily the quotient of the separately reported medians. The query column is dense/sparse, including each baseline count.

Workload	r	s	Queries	Dense	Sparse	Certified	Ratio
Coincident, static	12	2	2/2	13.127	0.076	0.169	173.50
Nested, static	12	13	13/13	12.760	0.499	1.060	25.59
Disjoint, static	12	13	4096/79	261.812	8.470	11.215	30.82
Disjoint, period 17	12	13	4096/79	252.919	3.199	4.520	78.59
Disjoint, reflection	12	13	4096/79	336.801	9.578	13.041	35.13
Overlapping, static	8	16	52/43	1.369	0.835	1.536	1.64
Full support, $r = 3$	3	8	8/8	0.139	0.175	0.429	0.79
Full support, $r = 4$	4	16	16/16	0.646	0.810	1.528	0.80

Table 1: Completed interval-subsystem benchmarks. Times are milliseconds; “Certified” includes sparse production and independent replay. Ratios below one are regressions. Generated from `results/benchmark.json`.

Coincident ports show the largest ratio, about 173.5, while both arms make only two orbit calls. The gain comes from avoiding the 2^{12} array and subset-processing work, not from a faster orbit kernel. Nested ports similarly retain 13 calls in both arms but improve by about 25.6.

The three disjoint 12-port families reduce the call count from 4,096 to 79. Their paired ratios are approximately 30.8, 78.6, and 35.1 for static, period-17, and reflection systems. Those different ratios are expected: coned queries have different local reduction costs, and total call count alone does not predict total bit work. The overlapping case has a more modest ratio of about 1.64.

The two full-support controls are slower: dense/sparse paired ratios are about 0.794 and 0.799. Thus the sparse implementation takes roughly 25–26% more time on those cases. Both methods make the same number of orbit calls. The cost comes from sparse recovery’s additional bookkeeping, not from a change in the invariant being computed.

The identical sparse-control/sparse median paired ratios range from 0.948 to 1.015 across the eight workloads. The static disjoint case therefore retains a roughly five-percent control discrepancy. The large speed differences and exact query reductions are robust observations in this run; fine-grained percentage claims are not justified by these few samples. No confidence interval or universal hardware-independent speedup is claimed.

11.4 Scale, certificates, and representation effects

Ports	r	Bits	s	Queries	Sparse	Certified	Proof bytes
Disjoint	32	501	33	529	341.859	420.231	144,591
Nested	64	501	65	65	47.669	78.421	110,660
Coincident	128	16001	2	2	4.558	6.357	584,103

Table 2: Sparse-only scale cases. “Bits” is `size.bit_length()`. Times are milliseconds; proof size is compact exact JSON. No dense timing was run for these rows.

The 32-port disjoint case has 33 positive records and 529 orbit calls, compared with an analytical dense output size of 2^{32} slots. This is not an experimental dense/sparse timing ratio: the dense run was not made. The 64-port nested case uses 65 calls and 65 positive records, despite its 2^{64} possible subset masks. The 128-port coincident case has just two positive records and two orbit calls on a 2^{16000} -point universe.

Small combinatorial output does not guarantee tiny serialized evidence. The last row’s certificate is 584,103 bytes in compact hexadecimal-safe JSON, because input bindings and large endpoints are explicit. At 12 ports, the three disjoint certificates retain 23 coned traces despite 78 coned search queries, with sizes 28,415, 19,408, and 41,085 bytes respectively. At 32 disjoint ports, 63 coned traces remain from 528 coned search queries. These figures illustrate the difference between logical queries, distinct queried unions, retained proofs, and physical serialized bytes.

11.5 What the experiments do not establish

These measurements do not show faster unknot decisions, improved recognition coverage, or better asymptotic dependence on crossing number. They do not exercise an attachment-port extractor from native triangulations. They also do not compare against a complete independently implemented weighted AHT engine. They establish correctness and measured costs for the supplied interval-relation primitive against the repository’s actual dense baseline. The mathematical polynomial conclusion comes from the proofs and classical AHT estimates, not from extrapolating the timing table.

12 The remaining route to quasi-polynomial recognition

12.1 A compositional bound with named obligations

Let n denote the size of a knot diagram or another explicitly specified recognition input. Lackenby's iteration analysis bounds the number of passes through a step by $L(g + 1)^L$, in terms of maximal hierarchy length L and maximal pattern complexity g [2, Proposition 9.1]. The following bookkeeping statement identifies what the local improvement would contribute to such an implementation.

Proposition 12.1 (Conditional end-to-end accounting). *Suppose a correct and complete implementation has at most a constant multiple of $L(g + 1)^L$ iterations. Suppose all states have binary length at most $\Sigma(n)$; constructing every interval system and its explicit ports costs $\text{poly}(\Sigma(n))$; each iteration performs only $\text{poly}(\Sigma(n))$ incidence queries; and all other work per iteration is at most $A(n)$. Then replacing dense incidence by this sparse primitive gives a total bound of the form*

$$O\left(L(g + 1)^L (\text{poly}(\Sigma(n)) + A(n))\right). \quad (9)$$

In particular, $L = O(\log n)$, $g \leq n^{O(1)}$, $\Sigma(n) \leq n^{O(\log n)}$, and $A(n) \leq n^{O(\log n)}$ imply $n^{O(\log n)}$ total time.

Proof. Each supplied interval system has k, M, r, B bounded polynomially by its explicit state and construction length. Corollary 5.2 and the assumed number of queries absorb all incidence work into one polynomial in $\Sigma(n)$. Add the other per-iteration work and multiply by the iteration bound. Under the final hypotheses,

$$\log(L(g + 1)^L) = O(\log \log n) + O((\log n)^2),$$

and a fixed polynomial in a quasi-polynomial state bound remains $n^{O(\log n)}$. Multiplication preserves that form. $\square$

This proposition is conditional on explicitly named geometric and algorithmic statements. It is not a proof that they hold. In particular, polynomial-time checking of a supplied compressed surface is not polynomial-time discovery of the required surface. A component histogram is not a compressed cutting operation. The counterexample in Section 9 rules out one tempting way to conflate them.

12.2 Where the remaining difficulty is located

The new primitive removes the need to demand logarithmically many ports solely to justify Boolean subset enumeration. It does not remove the need to keep the *explicit total number of port pieces* under control. Allowing a port to be specified by a more succinct grammar would create a different input model: the present theorem cannot be applied by charging only the grammar length while quietly expanding its interval list.

Nor does an iteration bound in terms of L and g automatically become a quasi-polynomial bound in n . If L is linear in a polynomial-size state, then even polynomial g can leave $L \log(g + 1)$ much larger than $O((\log n)^2)$. The required progress may come from stronger geometric bounds, accelerated batches of hierarchy operations, or a different potential function. None follows from sparse incidence alone.

For a serious integration effort, the next proof contracts should bind a diagram to its exterior, the exterior to encoded surfaces and ports, and each cutting or attachment step to its resulting state. The bit-size recurrence and the iteration potential must then be proved for the same implementation that emits those certificates. A collection of individually correct local primitives is necessary evidence, but not sufficient end-to-end complexity accounting.

13 Recommended changes to the maintained analysis

13.1 Clarify representation rather than retract the dense theorem

The existing synthesis correctly states that its dense histogram has 2^r entries, that union reuse cannot remove that output cost, and that weighted AHT offers another route [6]. No correction to the coning identity or dense inversion is needed. A suitable follow-up is an explicit companion statement: when only positive signatures are requested, the sparse API removes subset enumeration and is polynomial for all explicit interval-union inputs, by the reduction and classical support estimate in this report.

The older sufficient restrictions on r should remain attached to the *dense* query theorem. They should not be presented as an intrinsic barrier for all marked-incidence algorithms. Conversely, removing those restrictions from an aggregate query must not be advertised as removing them from every geometric state or attachment operation.

13.2 Adopt a separate sparse result type

A sparse result should specify that an absent mask means zero and that an unmarked entry may be present. Consumers should not use list position as a mask, assume that the counts are all one, or omit the unmarked term. The signed result similarly gives two counts per present mask, not a Boolean orientability flag for the whole system. These distinctions are covered by the included examples and tests.

The recommended initial integration is additive: copy the overlay, run its focused tests and the full target suite, and expose the sparse API to research callers. Keep the old dense API for consumers that genuinely need every array position and for small-instance cross-checks. Do not change ordinary knot dispatch merely because an isolated interval benchmark has a large speed ratio.

13.3 Keep evidence categories separate

The native source snapshots are pinned exactly. The sparse algorithm and certificate theorems are proved in the text. The test log gives executed case coverage, and the benchmark files give measured component costs. Neither substitutes for the other. Any future summary should preserve the negative full-support timings and the distinction between source-bound interval proofs and geometric provenance.

14 Further research questions and proposed deliverables

1. **A sharper support bound.** Can the factor Q multiplying the worst-case active weight partition in (5) be eliminated, perhaps by accounting for active and completed breakpoint records together? A useful deliverable is a proved bound of order $M + \text{poly}(k, B)$, or a compact family forcing a larger order. The proof must count output records with multiplicities, not just active weight values.
2. **Attachment-compatible compressed states.** What is the smallest additional data that defeats Proposition 9.1 while remaining closed under the actual hierarchy gluing operations? Start with explicit source-position correspondences and require a verifier for the induced equivalence relation after attachment. A successful result needs both an update algorithm and a worst-case size recurrence; another aggregate histogram without transport data does not meet the target.
3. **Native geometric provenance.** Construct a proof-carrying map from a knot diagram to a triangulated exterior, then from a supplied normal surface to its interval system, parity character,

and marked ports. The immediate experimental deliverable is a paired benchmark on genuine adapter output, checked independently against small expanded surfaces. This is the missing bridge between the present subsystem timings and a claim about recognition performance.

4. **Fewer unweighted queries and faster residual sums.** The rs logical bound is deliberately elementary. Can query patterns exploit port containments, equal coned inputs, or the endpoint structure without assuming constant hypergraph rank? Can subset sums over recovered atoms be indexed in a way that beats the current $O(rs^2)$ scan while retaining a simple proof contract? Compare total bit work, not just oracle call count: the full-support regressions show that these objectives can differ.
5. **A certified weighted kernel comparison.** Implement the classical weighted AHT transfer operation with independent weight-preservation certificates, including binary multiplicities and vector-valued weights. Compare that direct method against repeated unweighted coning on the same supplied systems. This is not a proposal to rediscover the weighted polynomial theorem; it is a proposal to determine which proof-carrying implementation is preferable in practice.
6. **Proof minimization and reusable facts.** The singleton family reduces quadratic search queries to only $2r - 1$ retained coned proofs. Characterize when much more can be discarded, and whether a small collection of dominance-zero facts can certify several atoms simultaneously. Any proof-sharing scheme must preserve source and union bindings, including after ports are renamed or merged.
7. **An implementation-level hierarchy potential.** Establish a bound on the sum of logarithmic costs of the actual hierarchy operations, rather than separately bounding only their number and nominal size. The target is a proved inequality yielding $O((\log n)^2)$ in the exponent of (9), with explicit charges for cutting maps, coefficient growth, retries, and search branches. This is a central remaining route to the requested global complexity.
8. **Formalize the monotone certificate core.** The sparse certificate theorem uses finite sets, nonnegative integer sums, and induction; it is substantially smaller than a full formalization of normal-surface topology. A proof-assistant development could first verify Theorem 6.1 and the cone identity, then link an executable checker to those statements. The current tests and handwritten proof are not a substitute for that development.
9. **End-to-end ablation without optimistic censoring.** After provenance and attachment support exist, measure complete recognizer runs with and without sparse incidence under shared resources. Retain knotted and unknotted inputs, easy controls, failures, and resource-limited runs as separate outcomes. Report a recognition speedup only when the whole run completes with independently checked evidence; do not convert a local capacity increase into a fictitious timing ratio.

15 Reproducibility and delivery contents

The archive is self-contained for the new interval tests and benchmarks; it is not a copy of the complete ProveIt repository. Its `reference/` directory contains the four exact upstream dependencies. The `overlay/` directory contains only new implementation and test files for copying beneath `Topology/UnknotRecognition`. The bootstrap used by standalone experiments adds the overlay modules to the reference package’s search path without modifying its source.

From the archive root, run:

```
python experiments/run_tests.py
python experiments/benchmark.py --rounds 7 --scale-rounds 3
python experiments/make_tables.py
python experiments/make_examples.py
sh article/build.sh
```

The benchmark regenerates raw samples and tables; timing values naturally vary. `results/benchmark.json` contains the final complete measurement set and source hashes. `benchmark_samples.csv` exposes every arm-round record. The separate timeout-attempt log is historical evidence, not part of that dataset. The tests check the upstream Git blob identifiers before executing any cases. `examples/` supplies unsigned and signed certificates and the four-point gluing obstruction as exact JSON.

The `README.md` explains integration, scope, API usage, and the known limitations. `RESEARCH_STATUS.md` separates proved claims, inherited results, executed experiments, and unresolved obligations. `SOURCE_MANIFEST.json` records provenance, and `SHA256SUMS.txt` covers the delivered files. No repository write or pull request is performed by this delivery.

16 Conclusion

The dense marked-incidence interface pays for the Boolean lattice even when the represented component data is small. Nonnegative residual recovery removes that cost while supplying a sparse, independently checkable completeness argument. Classical weighted AHT theory then upgrades the output-sensitive bound to a polynomial bound for every explicit interval-union input; the proof does not need a logarithmic cap on the number of ports. Exact query formulas and completed measurements show where the improvement comes from, while full-support controls expose its constant-factor regressions.

The resulting code is a concrete component for a compressed hierarchy, not a new unknot recognizer. A source-bound interval certificate says what it proves and no more. The immediate mathematical next step is to retain attachment-compatible information with comparable size and verification bounds. The broader target still requires geometric provenance, complete search, and an implementation-level hierarchy complexity argument.

References

- [1] I. Agol, J. Hass, and W. Thurston, *The computational complexity of knot genus and spanning area*, Transactions of the American Mathematical Society **358** (2006), 3821–3850. Preprint version used: arXiv:math/0205057v2, especially Theorem 12, Lemma 15, and Theorem 16. <https://arxiv.org/abs/math/0205057>.
- [2] M. Lackenby, *Incompressible surfaces, hierarchies and unknot recognition*, arXiv:2607.23350v1 (2026), especially Sections 9–10. <https://arxiv.org/abs/2607.23350>.
- [3] N. H. Bshouty and H. Mazzawi, *Optimal query complexity for reconstructing hypergraphs*, arXiv:1001.0405 (2010). <https://arxiv.org/abs/1001.0405>.
- [4] ProveIt project, pinned snapshot specified on the title page. Within `Topology/UnknotRecognition/fast/fastunknot/`: `interval_orbits.py` and `interval_orbit_verify.py`. Exact copies and hashes accompany this report. <https://github.com/VladimirReshetnikov/ProveIt>.
- [5] ProveIt project, same snapshot and module directory: `interval_incidence.py`. Exact dependency hash: see `SOURCE_MANIFEST.json` in the delivery.
- [6] ProveIt project, same snapshot, within `Topology/UnknotRecognition/`: `synthesis/orbit_certificates.tex` and `synthesis/README.md`.